

Universidad Tecnológica Nacional – Facultad Regional Rosario
Ingeniería en Sistemas de Información
Cátedra de Redes de Datos

CAPA DE TRANSPORTE
SERVICIOS Y PRIMITIVAS

CAPA 4 – SERVICIOS Y PRIMITIVAS DE TRANSPORTE

Esta capa, junto a la capa 3, constituye el corazón de la jerarquía de protocolos OSI.

- **ÍNDICE:**

1 – Introducción.

2 – Servicios con o sin conexión.

3 – Segmentos.

4 – Primitivas de transporte.

5 – Sockets de TCP.

6 – Programación de sockets.

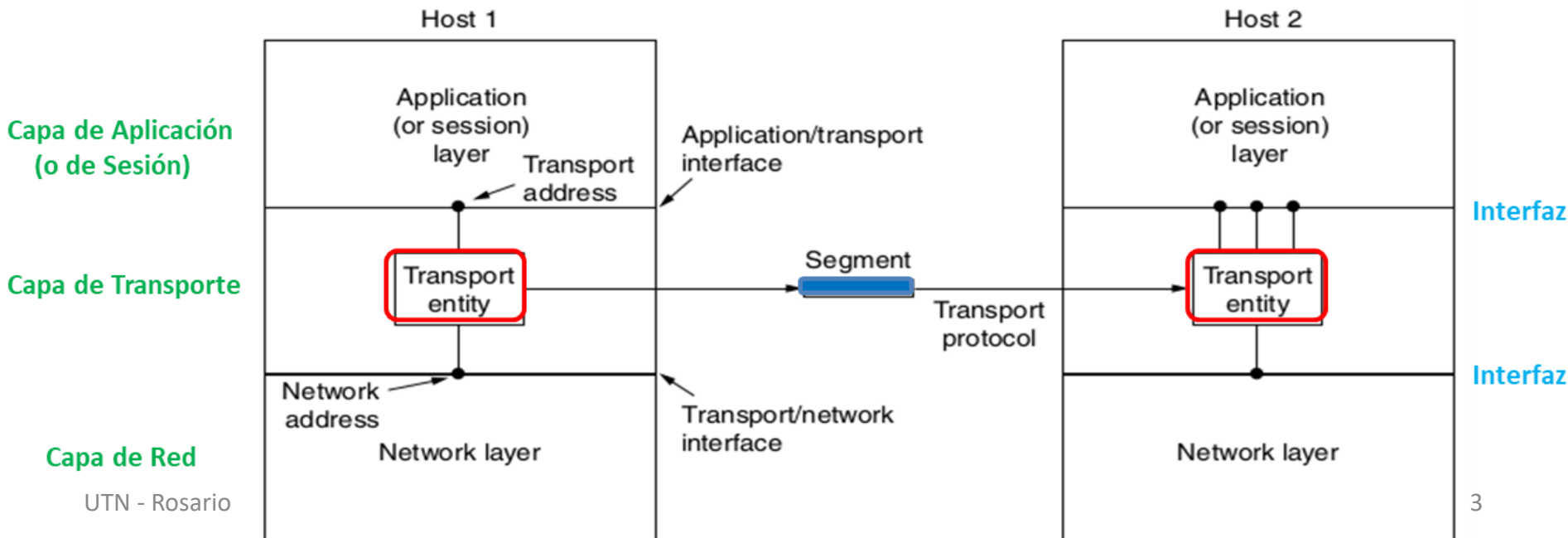
7 - Bibliografía.

CAPA DE TRANSPORTE DEL MODELO OSI



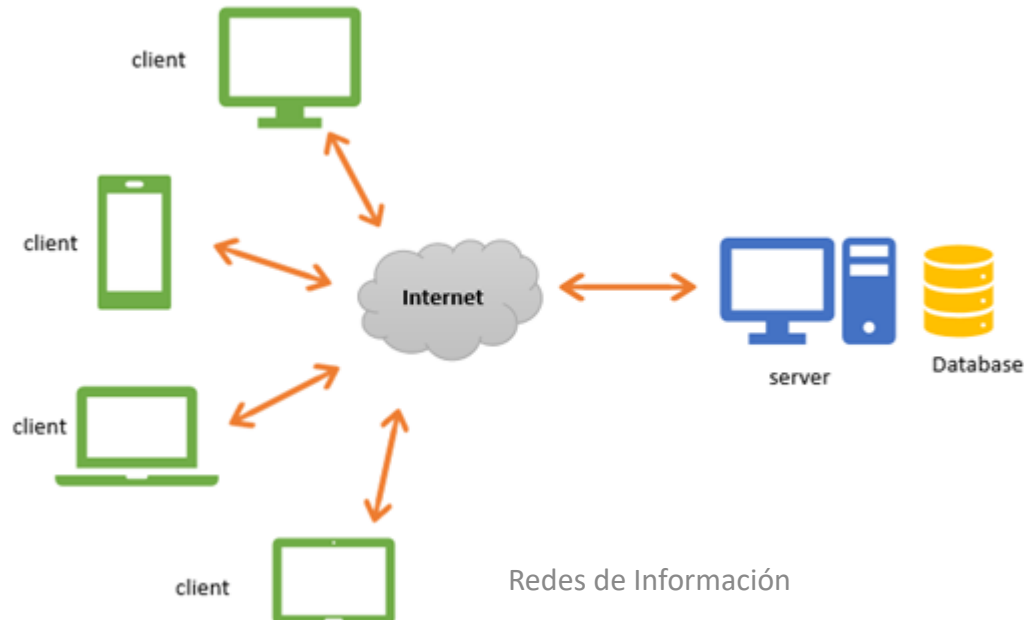
1- INTRODUCCIÓN.

- **Objetivo:** El fin de la Capa de Transporte es brindarle un servicio de transmisión de datos, confiable, eficiente y costo-efectivo a la Capa superior de Aplicación.
- **Entidad de transporte:** Al conjunto de software y hardware que constituye la capa 4 se lo denomina “Entidad de transporte”, y puede estar localizado en varios lugares: en el Kernel del sistema operativo, o bien, en paquetes de librerías con aplicaciones de red, o bien, en una placa de red, o incluso, en un proceso separado de un usuario.
- **De extremo a extremo:** Observando el modelo OSI desde las capas más bajas, la Capa de Transporte es la primera que opera de “extremo a extremo”, sin importar la cantidad de redes que existan en el camino. A diferencia de las capas 1,2 y 3 que trabajan “salto a salto”, o “punto a punto”. Así como los routers son los elementos principales de esa capa, donde reside el hardware y software principal, en la capa 4 nos encontramos que los usuarios o hosts son los principales elementos donde reside el software y hardware de transporte.
- Las relaciones lógicas entre la capa de red, de transporte y de aplicación, se observan en la figura:



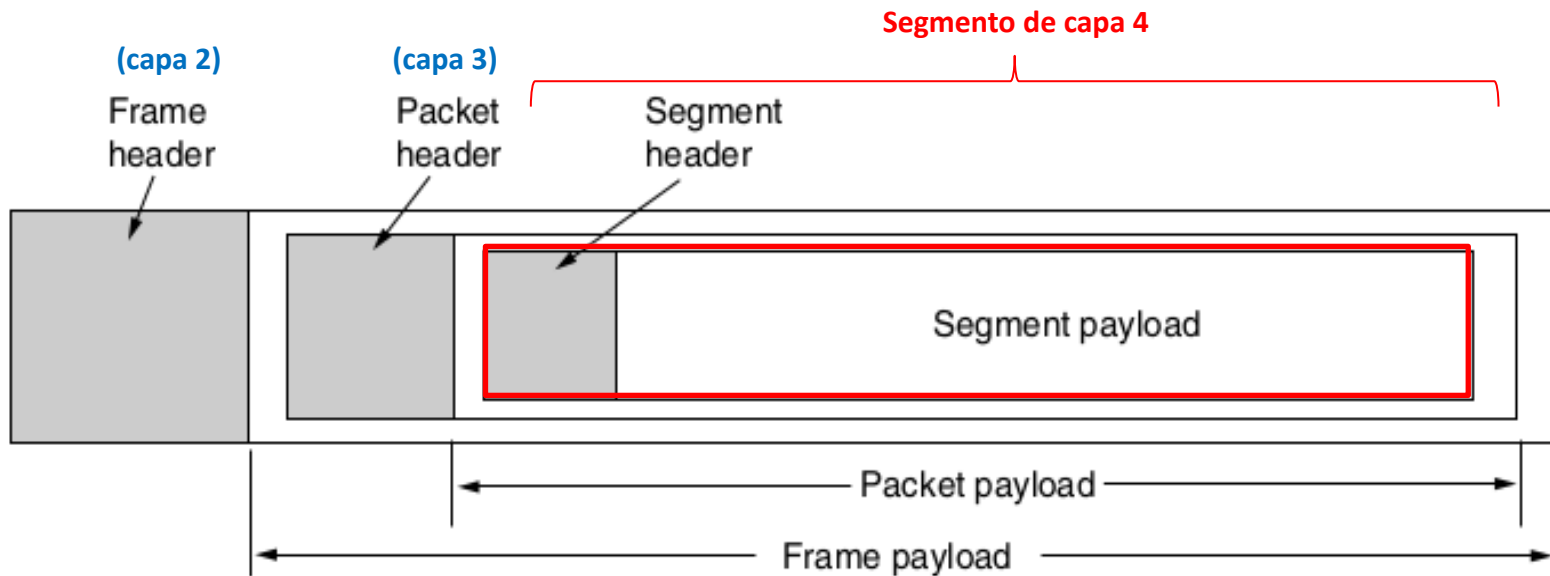
2 – SERVICIOS.

- Al igual que en Capa 3, existen dos tipos de servicios diferentes en Capa 4:
- ✓ **Servicio Orientado a Conexión:** El servicio de transporte orientado a conexión brinda conexiones de datos confiables, a pesar de que las redes son imperfectas, pierden paquetes, sufren congestiones y otras anomalías. Este servicio de capa 4 se encarga de la retransmisión de paquetes, por ejemplo, para lograr una conexión confiable entre origen y destino. Tiene tres fases bien diferenciadas que son Establecimiento, Transferencia de datos y Liberación. Y donde también tenemos procesos de control de flujo y direccionamiento de los paquetes. Aquí se utiliza el protocolo TCP que realiza control de errores y retransmisión de segmentos si fuera necesario.
- ✓ **Servicio no Orientado a Conexión:** El servicio de transporte también puede brindarse sin conexión ni confiabilidad, simplemente como datagramas. Por ejemplo, se emplea en la práctica para streamings de audio y video, donde no hay tiempo para retransmisiones de datos en tiempo real. Es un servicio simple, sin demasiados controles, y que es implementado con el protocolo UDP. El control de errores no es obligatorio sino optativo.
- **Cliente-servidor:** En Capa de Transporte es común ver una arquitectura del tipo “cliente-servidor”, como la de la figura:



3 - SEGMENTOS.

- En capa 3 llamamos “paquete” a la unidad de datos, aquí en capa 4 lo denominamos “segmento” al mensaje de capa de transporte. Antiguamente se lo llamaba TPDU (Transport Protocol Data Unit). Ahora simplemente se le dice “segmento”. En la figura se ilustran las tramas, paquetes y segmentos anidados de capas 2,3 y 4.



Nesting of segments, packets, and frames.

4 - PRIMITIVAS.

- **“Sockets”**: Las primitivas son unidades de software que realizan una pequeña función específica, como parte del servicio de esa capa. En general, los programadores de la capa de transporte escriben sus primitivas independientemente de la red donde va a funcionar. En capa 4 se las conoce como “sockets”.
- Ejemplo: Primitivas cliente-servidor: En la tabla se observan 5 primitivas. Consideremos el ejemplo de una aplicación donde interviene un servidor y varios clientes remotos. En primer lugar, el servidor ejecutaría un **“LISTEN”**, es decir, llamaría a un programa de librería que “escucha” y bloquea al servidor hasta que algún cliente se quiera comunicar. Cuando eso suceda, ejecutaría la primitiva **“CONNECT”**. Y luego se pasaría a la primitiva **“SEND”** donde se transmiten los datos desde el servidor, etc.

Primitivas utilizadas en
arquitectura cliente-servidor

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	Request a release of the connection

5 – SOCKETS de TCP.

- Las primeras “sockets” de TCP aparecieron en 1983 con la versión de Unix “Berkeley 4.2 BCD” y se popularizaron enseguida. En el sistema operativo Windows, existe un API (Application Programming Interface) similar y las primitivas se llaman “winsock”.
- En arquitectura cliente-servidor, las 4 primeras primitivas para TCP son ejecutadas en este orden por los servidores:
 - 1- SOCKET:** Crea un extremo de comunicación y reserva espacio en las tablas de la entidad de transporte.
 - 2- BIND:** Asigna direcciones locales de capa 4 al socket, para que los clientes se puedan comunicar, (puertos).
 - 3- LISTEN:** Asigna espacio de almacenamiento para colas de espera de clientes que quieran comunicarse con el server.
 - 4- ACCEPT:** Se utiliza cuando llega un segmento de cliente solicitando conexión.
- Al finalizar la conexión se ejecuta un “CLOSE” de ambos lados. El fin de la comunicación es simétrico.
- Del lado del cliente, las primitivas son distintas, aunque parecidas. Por ejemplo “BIND” no es necesario ya que el servidor no se fija en la dirección del cliente.

Primitivas TCP

Primitive	Meaning
SOCKET	Create a new communication endpoint
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

Crea un extremo de comunicación y reserva espacio

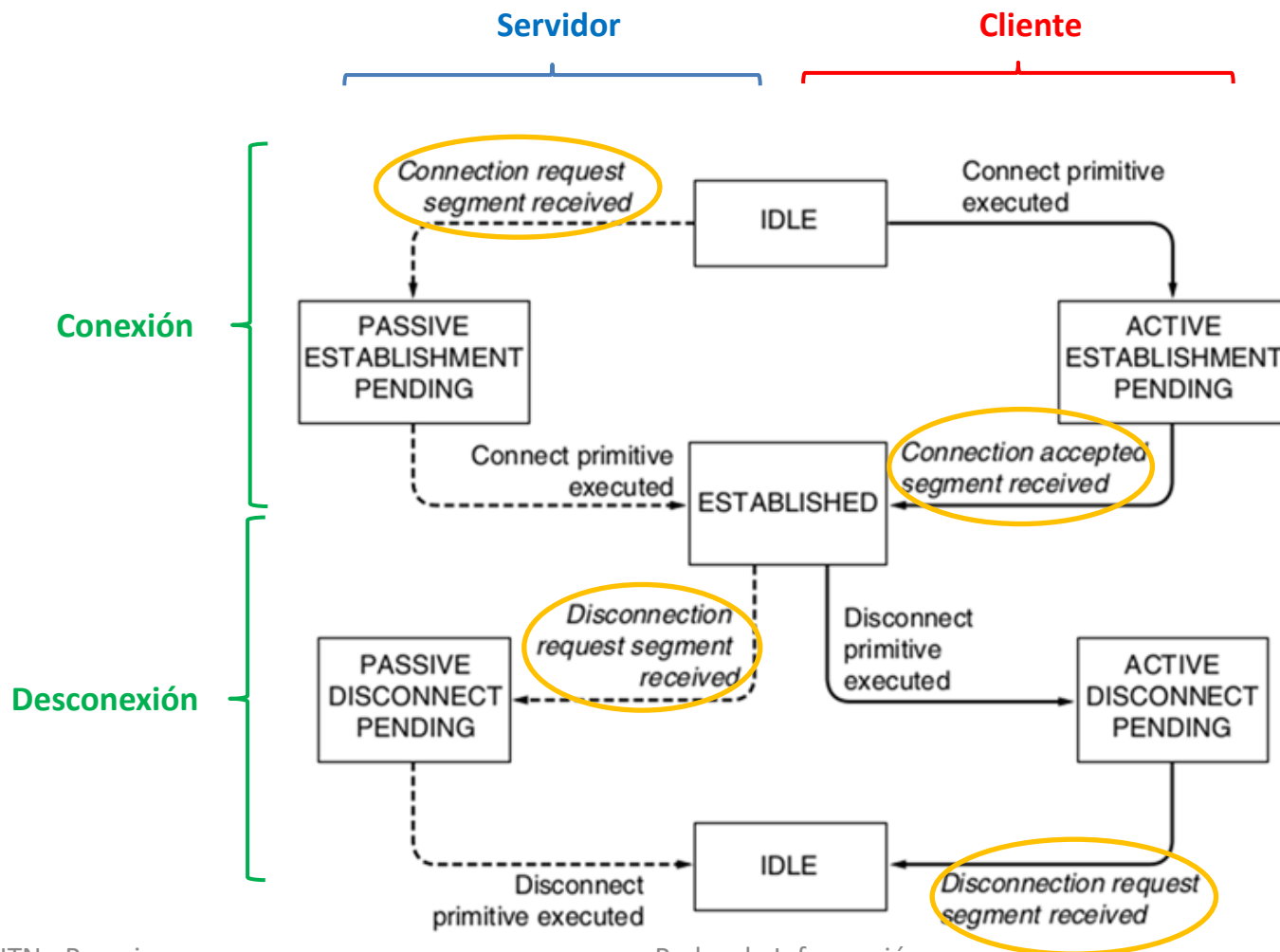
Asigna un puerto de capa 4 al socket.

Asigna espacio para colas de espera de clientes.

Se utiliza cuando llega un segmento solicitando conexión.

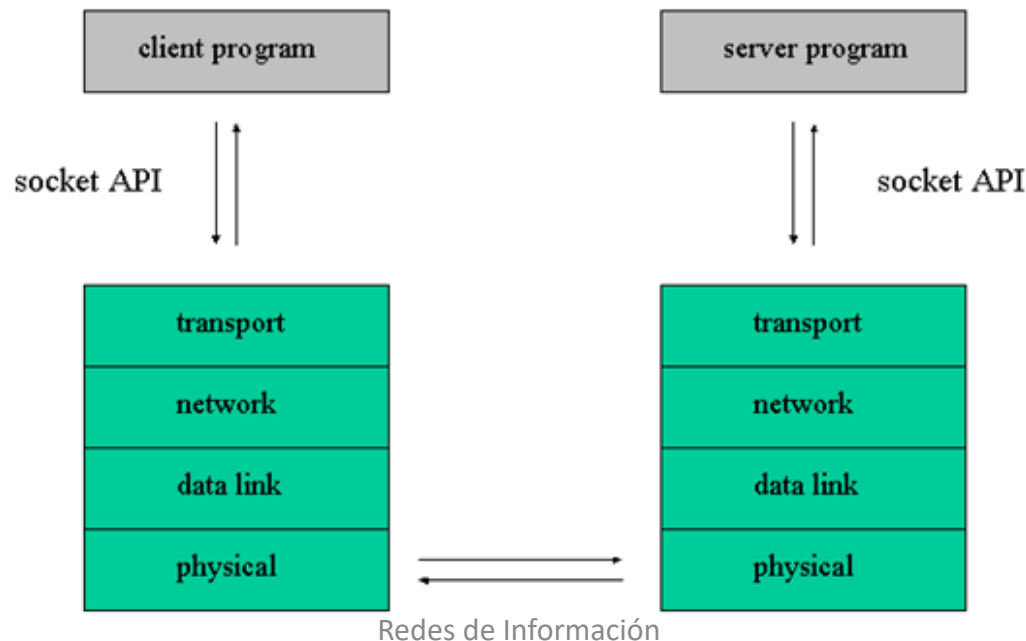
5 - SOCKETS (continuación).

- **Diagrama de Estados:** En la figura se observa un “diagrama de estados” de un esquema de conexión simple en arquitectura cliente-servidor.
 - Las letras *itálicas* corresponden al arribo de un paquete.
 - Las líneas continuas muestran la secuencia de estados del cliente.
 - Las líneas de trazos ilustran la secuencia del servidor.



5 - SOCKETS (continuación).

- **Socket orientado a conexión:** Los sockets son muy populares. El socket API se utiliza en TCP para una comunicación “orientada a conexión” llamada “Reliable Byte Stream”.
- **Socket sin conexión:** Los sockets también pueden ser utilizados por aplicaciones para “comunicaciones no orientadas a conexión”. En este caso, la primitiva “CONNECT” define la dirección del lado remoto y “SEND” y “RECEIVE” se encargan de comunicar datagramas entre ambas partes o “peers”.
- Nuevos protocolos se han creado desde la aparición de las primeras “sockets” y el tiempo dirá si son los más efectivos para ciertas aplicaciones o cambiarán en el futuro. Estos protocolos cambian levemente las “sockets” API estándar. Por ejemplo:
 - SCTP, (Stream Control Transmission Protocol), definida en RFC 4960.
 - SST, (Structured Stream Transport) diseñada por Ford, en 2007.



6 - PROGRAMACIÓN DE SOCKETS.

Ejemplo: Servidor de archivos de internet – lado del servidor.

```
#include <sys/types.h>
#include <sys/fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345
#define BUF_SIZE 4096
#define QUEUE_SIZE 10

int main(int argc, char *argv[])
{
    int s, b, l, fd, sa, bytes, on = 1;
    char buf[BUF_SIZE];
    struct sockaddr_in channel;

    /* Build address structure to bind to socket. */
    memset(&channel, 0, sizeof(channel));
    channel.sin_family = AF_INET;
    channel.sin_addr.s_addr = htonl(INADDR_ANY);
    channel.sin_port = htons(SERVER_PORT);

    /* Passive open. Wait for connection. */
    s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP); /* create socket */
    if (s < 0) fatal("socket failed");
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, (char *) &on, sizeof(on));

    b = bind(s, (struct sockaddr *) &channel, sizeof(channel));
    if (b < 0) fatal("bind failed");

    l = listen(s, QUEUE_SIZE); /* specify queue size */
    if (l < 0) fatal("listen failed");

    /* Socket is now set up and bound. Wait for connection and process it. */
    while (1) {
        sa = accept(s, 0, 0); /* block for connection request */
        if (sa < 0) fatal("accept failed");

        read(sa, buf, BUF_SIZE); /* read file name from socket */

        /* Get and return the file. */
        fd = open(buf, O_RDONLY); /* open the file to be sent back */
        if (fd < 0) fatal("open failed");

        while (1) {
            bytes = read(fd, buf, BUF_SIZE); /* read from file */
            if (bytes <= 0) break; /* check for end of file */
            write(sa, buf, bytes); /* write bytes to socket */
        }

        close(fd); /* close file */
        close(sa); /* close connection */
    }
}
```

(1) →

(2) →

(3) →

(4) →

(5) →

6 - PROGRAMACIÓN DE SOCKETS (continuación).

- Ejemplo: “Servidor de archivos de internet” - lado del servidor.
 1. Después de declarar variables locales, comienza inicializando una estructura de datos que contendrá la dirección IP del servidor y quedará atada al socket del servidor. (“**MEMSET**” coloca ceros en toda la estructura de datos).
 2. “**HTONL**” y “**HTONS**” son funciones que convierten los valores a un formato estándar para asegurar que funcione bien en máquinas pequeñas como Intel x86 y también en grandes como la SPARC. Luego el servidor crea un socket y chequea errores, indicado por $s < 0$.
 3. “**SETSOCKOPT**” permite reusar el mismo puerto del servidor en un nuevo cliente. Nuevamente se asocia la dirección IP al socket y se chequea con “**BIND**”.
 4. “**LISTEN**” es el último paso, donde el servidor queda esperando un nuevo cliente. Si se superponen varios clientes, quedarán en cola de espera hasta un límite que, de excederse, simplemente se descartará el pedido de conexión.
 5. Y en este punto el servidor permanece en el bucle principal. “**ACCEPT**” bloquea al servidor hasta que aparezca algún cliente. “**ACCEPT**” devuelve un archivo descriptor bidireccional, es decir, que puede ser utilizado tanto para lectura como para escritura también. A diferencia de los “pipe”, por ejemplo, que son unidireccionales, aquí vemos que los sockets son bidireccionales.
- ✓ Apenas se acepta la conexión, el servidor lee el nombre del archivo, lo abre, y alternativamente lee y escribe bloques en el socket hasta que el archivo entero se haya copiado. Luego el servidor cierra el archivo y la conexión, y queda a la espera de una nueva conexión. Y así, continuamente.

6 - PROGRAMACIÓN DE SOCKETS (continuación).

Ejemplo: Servidor de archivos de internet – Lado del cliente.

```
/* This page contains a client program that can request a file from the server program
 * on the next page. The server responds by sending the whole file.
 */

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345 /* arbitrary, but client & server must agree */
#define BUF_SIZE 4096 /* block transfer size */

(1) int main(int argc, char **argv)
{
    int c, s, bytes;
    char buf[BUF_SIZE]; /* buffer for incoming file */
    struct hostent *h; /* info about server */
    struct sockaddr_in channel; /* holds IP address */

    if (argc != 3) fatal("Usage: client server-name file-name");
    h = gethostbyname(argv[1]); /* look up host's IP address */
    if (!h) fatal("gethostbyname failed");

    (2) s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (s < 0) fatal("socket");
    memset(&channel, 0, sizeof(channel));
    channel.sin_family = AF_INET;
    memcpy(&channel.sin_addr.s_addr, h->h_addr, h->h_length);
    channel.sin_port = htons(SERVER_PORT);

    (3) c = connect(s, (struct sockaddr *) &channel, sizeof(channel));
    if (c < 0) fatal("connect failed");

    /* Connection is now established. Send file name including 0 byte at end. */
    write(s, argv[2], strlen(argv[2])+1);

    /* Go get the file and write it to standard output. */
    while (1) {
        bytes = read(s, buf, BUF_SIZE); /* read from socket */
        if (bytes <= 0) exit(0); /* check for end of file */
        write(1, buf, bytes); /* write to standard output */
    }

    (4) fatal(char *string)
    {
        printf("%s\n", string);
        exit(1);
    }
}
```

Client code using sockets.

6 - PROGRAMACIÓN DE SOCKETS (continuación).

- **Ejemplo:** “Servidor de archivos de internet”- lado del cliente. (Ver el software en la diapositiva siguiente).
- Suponiendo que el programa se llame “client”, usualmente se ejecuta con “client flits.cs.vu.nl /usr/tom/filename >f”. Esto funciona siempre que el servidor ya se encuentre ejecutando “flits.cs.vu.nl”, y el archivo “/usr/tom/filename” exista, y además el servidor tenga acceso a él. Si la llamada es exitosa, el archivo se transfiere y se escribe en “f”, finalizando el programa “client”. El servidor queda activo y por lo tanto el cliente puede iniciar nuevas comunicaciones y transferencias con él, de ser necesario.
1. El código cliente comienza con algunas sentencias “**INCLUDE**” para utilizar librerías y “**DEFINE**” para declarar variables. Luego chequea si ha sido llamado con el número correcto de argumentos, (argc = 3) para el nombre del programa y dos argumentos. Notar que “argv [1]” contiene el nombre del servidor, por ejemplo “flits.cs.vu.nl”, que es convertido a dirección IP por “**GETHOSTBYNAME**”. Esta función utiliza el servicio DNS, que estudiaremos en capa de aplicación.
 2. Luego se crea e inicializa un socket. A continuación, los clientes intentan conectar con el servidor a través del protocolo TCP, con “**CONNECT**”. Si el servidor está corriendo en la máquina con nombre adecuado, y tiene asociado el “número de puerto” y hay lugar en la cola de espera, entonces se establece la conexión.
 3. Luego el cliente envía el nombre del archivo, escribiendo en el socket. Luego, lee bloque a bloque el archivo del socket hasta terminar y lo copia a una salida. Y listo.
 4. “**FATAL**” imprime un mensaje de error. El software del servidor también tiene un “**FATAL**”, que no se ve en el ejemplo por falta de espacio.
- ✓ Este ejemplo es bien sencillo y bastante pobre. Su performance es baja porque ejecuta secuencialmente los pasos y además no emplea métodos de seguridad. Para más información sobre programación en sockets se puede consultar a Donahoo y Calvert, 2008 y 2009.

7 – BIBLIOGRAFÍA.

- Tanenbaum, Andrew S. y Wetherall, David J: “Computer Networks”. 5th edition. Pearson Education Inc.